Question Bank: PostgreSQL Query Performance & Indexing

QUESTIONS.MD | INTERVIEWER | 2026-09-02

Type ratio: `theory` 2 of 6 = **33%** (cap is 40%); applied types 4 of 6 = **67%** (floor is 60%).

Q-L1-01

THEORY · WEIGHT 2

> **ASK** A B-tree index exists on `orders(created_at)`. Which of these can it serve, and which can't it: a range filter on `created_at`, a sort by `created_at`, and a filter on `status`? Say why in each case.

TIME BUDGET 3 min

ANSWER SHAPE decides each of the three cases, and grounds the decision in the index being an ordered structure on that column.

EXPECTED KEY POINTS

- [3] The index is ordered by `created_at`, which is what makes range filters and sorts on it servable
- [2] Range filter: yes — an ordered structure locates a contiguous range
- [2] Sort: yes — the index can be walked in order, avoiding an explicit sort
- [3] Filter on `status`: no — `status` is not in the index; nothing orders or locates by it

MODEL ANSWER A B-tree on `created_at` stores entries ordered by that column. A range filter on `created_at` is served by descending to the range start and walking, because the ordering makes the range contiguous. A sort by `created_at` is served by reading the index in order rather than sorting the heap. A filter on `status` cannot be served at all: the column isn't in the index, so there is no ordering or locating structure for it.

ACCEPTABLE VARIANTS Candidates who note the sort case depends on the planner judging the index walk cheaper than a seq scan plus sort are correct — award the [2] and count it toward depth.

RED FLAGS "an index makes any query on that table faster"; treating the index as an unordered lookup table; claiming the `status` filter works "because the index has the row pointers".

PROBES

- "Take the `status` case — say what the database would have to do."
- "You said it can serve the sort. What property of the index makes that true?"

SCORING ANCHORS

- 100 — all three cases decided correctly and grounded in ordering
- 70 — all three correct, but reasoning is "it's indexed / it's not indexed" rather than ordering
- 40 — gets the range case, treats the index as a generic speed-up for the rest
- 0 — no answer, or claims the index serves all three

DEPENDS ON none

Q-L1-02

THEORY · WEIGHT 2

> **ASK** You have an index on `orders(customer_id, created_at)`. A query filters only on `created_at`. Will the index be used? Explain.

TIME BUDGET 3 min

ANSWER SHAPE answers no (or "not as an ordinary index scan"), and explains it in terms of the leading column.

EXPECTED KEY POINTS

- [4] A composite B-tree is ordered by the leading column first; without a predicate on `customer_id`, entries matching a `created_at` range are scattered rather than contiguous
- [2] So it can't be used for an ordinary index scan on `created_at` alone
- [1] A separate index on `created_at`, or reordering the composite, would serve it

MODEL ANSWER No — not as an ordinary index scan. The index is ordered by `customer_id` first and only by `created_at` within each `customer_id`. Rows for a given `created_at` range are therefore scattered across the whole index rather than sitting in one contiguous run, so there is nothing to descend to. You'd need an index leading with `created_at`.

ACCEPTABLE VARIANTS Mentioning that PostgreSQL can still choose a full index scan or index-only scan when the index is much narrower than the heap is correct and is a depth signal — but only once they have established the leading-column point.

RED FLAGS "yes, because `created_at` is in the index"; "column order doesn't matter, the planner sorts it out".

PROBES

- "Describe how the entries are laid out in that index."
- "You said it's in the index. Where would the database start looking?"

SCORING ANCHORS

- 100 — correct answer, grounded in leading-column ordering, plus the fix
- 70 — correct answer and leading-column reasoning, no fix offered
- 40 — says no, but the reason is a recalled rule rather than the ordering
- 0 — says yes because the column appears in the index

DEPENDS ON Q-L1-01

Q-L2-01

DEBUG · WEIGHT 3

> **ASK** There's a B-tree index on `orders(created_at)`. This query is doing a sequential scan over 80 million rows: `SELECT * FROM orders WHERE date(created_at) = '2026-01-15'`. The index is valid and the table was analyzed this morning. Why isn't the index used, and what would you change?

TIME BUDGET 5 min

ANSWER SHAPE names the cause, explains why it defeats the index, gives a fix, and states the fix's cost or condition.

EXPECTED KEY POINTS

- [4] The predicate wraps the indexed column in a function, so the condition isn't on `created_at` — it's on `date(created_at)`, which nothing indexes
- [2] The index's ordering is on the raw values; the function's output has no corresponding ordering to search
- [2] Fix: rewrite as a half-open range — `created_at >= '2026-01-15' AND created_at < '2026-01-16'` — which the existing index serves directly
- [1] Alternative: an expression index on `date(created_at)`, at the cost of another index to maintain on a table taking 8k inserts/sec

MODEL ANSWER The predicate applies `date()` to the indexed column, so the condition is on a derived value, not on `created_at`. The index is ordered by the raw timestamps and there is no ordering of `date(created_at)` to descend into, so the planner has no way to use it and falls back to a scan. The clean fix is a half-open range on the raw column. An expression index on `date(created_at)` would also work, but that's a second index on a write-heavy table, so the rewrite is preferable.

ACCEPTABLE VARIANTS Reaching the same place via "the predicate isn't sargable" is fully correct provided they then explain what that means here. `BETWEEN '2026-01-15 00:00:00' AND '2026-01-15 23:59:59'` earns the fix points — it works for timestamp but carries a boundary risk; a candidate who volunteers that risk shows depth.

RED FLAGS blaming stale statistics (the question rules it out — blaming it anyway means they aren't reading the constraints); "PostgreSQL doesn't index timestamps well"; proposing `SET enable_seqscan = off` as the fix.

PROBES

- "Walk me through what the database does with that WHERE clause."
- "You've named the cause. What would you actually change?"
- "You proposed an expression index. On this table, what does that cost?"

SCORING ANCHORS

- 100 — function-wrapping identified, ordering explained, range rewrite given, cost of the alternative named
- 70 — cause and fix correct, no cost discussion
- 40 — senses the predicate is the problem but can't say why, or offers only "add an index"
- 0 — blames statistics, hardware, or row count; or disables seqscan

Q-L2-02

PRACTICAL · WEIGHT 3

> **ASK** Here's a line from EXPLAIN ANALYZE on a query blowing the 200ms budget: `Index Scan using idx_orders_created_at on orders (cost=0.43..8.45 rows=1 width=248) (actual time=0.02..412 rows=2100000 loops=1)`. Tell me what this line says, and where you'd look next.

TIME BUDGET 5 min

ANSWER SHAPE reads the specific numbers, names the estimate-versus-actual gap as the finding, and gives a next step that follows from it.

EXPECTED KEY POINTS

- [4] Estimated 1 row, actually returned 2.1M — that six-order-of-magnitude misestimate is the finding
- [2] The planner chose an index scan assuming near-perfect selectivity; at 2.1M rows that choice is wrong, and every plan node above it was built on the same bad estimate
- [2] Next step: find why the estimate is off — correlated predicates, a skewed column distribution, insufficient statistics target, or an expression the planner can't estimate
- [1] Concretely: check `n_distinct` and the histogram for the column, consider extended statistics for correlated columns, or raise the statistics target

MODEL ANSWER The planner expected one row and got 2.1 million, and that misestimate is the whole problem — it picked an index scan because it thought the predicate was almost perfectly selective, and every choice above this node inherited the same assumption. Actual time running from 0.02 to 412ms is what that costs. I'd stop looking at the index and go after the estimate: check the column's statistics and distribution, look for correlated predicates the planner is multiplying into a near-zero selectivity, and consider extended statistics or a higher statistics target.

ACCEPTABLE VARIANTS Candidates who first ask whether this is the top node or a nested-loop inner side are reasoning well — answer "inner side of a nested loop", and expect them to add that the misestimate also produced a bad join strategy. That addition is a strong depth signal.

RED FLAGS reading `cost=0.43..8.45` as milliseconds; concluding "the index is bad, drop it"; proposing another index without addressing the estimate; ignoring the rows discrepancy entirely.

PROBES

- "Compare the estimate and the actual. What do you make of the difference?"
- "You've read the timing. What else on that line is telling you something?"
- "You want to change the index. Why did the planner pick this one?"

SCORING ANCHORS

- 100 — misestimate identified as the finding, its consequence for the plan explained, statistics-directed next step given
- 70 — spots the estimate/actual gap and knows it matters, next step vague
- 40 — reads the timing correctly but treats the index as the problem
- 0 — misreads cost as time, or proposes index changes with no mention of the row gap

DEPENDS ON none

Q-L3-01

TRADEOFF · WEIGHT 4

> **ASK** The orders table takes about 8,000 inserts per second at peak. A team wants to add three new indexes to speed up their reporting queries. You have to give an answer. What is it, and what does it cost you?

TIME BUDGET 6 min

ANSWER SHAPE takes a position, names what the position turns on, and states the cost of the side they chose — not a survey of considerations.

EXPECTED KEY POINTS

- [3] Every additional index is maintained on every insert, so three indexes mean write amplification on a path already at 8k/sec — the cost lands on the write path, not the reporting path
- [3] Takes an actual position and names what it turns on: measured write headroom, whether reporting must be live on the primary, whether one composite index serves the queries instead of three
- [2] Names the cost of the chosen side honestly — slower inserts, more WAL and bloat; or slower/staler reports
- [2] Concrete alternative: serve reporting from a read replica or a materialized rollup, so the write path takes no new indexes
- [1] Secondary effects: more WAL, larger checkpoints, more autovacuum work, and lost HOT updates if the new indexes cover columns that get updated

MODEL ANSWER My answer is no to three indexes on that table as stated, and it turns on where the reporting reads have to run. Every index is maintained synchronously on every insert, so three of them multiply work on a path already doing 8k/sec — plus more WAL, bigger checkpoints, more vacuum, and lost HOT updates if any indexed column is ever updated. The cost of my "no" is real: reporting stays slow, and I'm accepting that deliberately. What I'd do instead is push reporting to a replica or a rollup, where indexes cost the write path nothing. If reporting genuinely must be live on the primary, I'd take one composite index covering the most queries, measure insert latency before and after, and go from there — not three at once.

ACCEPTABLE VARIANTS "Yes" is equally correct **if** the cost is paid honestly — they establish measured write headroom, commit to measuring insert latency, and name what they'd roll back on. The position doesn't matter; the cost does.

RED FLAGS "indexes only slow down writes a little"; recommending all three with no cost named; refusing to take a position even after the probe ("it depends", with nothing named that it depends on); proposing to drop indexes during business hours to test.

PROBES

- "Give me your answer. Yes or no, then justify it."
- "You've said what you'd do. What does it cost you?"
- "You've named the write cost. Is there a way to get the reporting speed without paying it?"

SCORING ANCHORS

- 100 — clear position, named dependencies, cost of their own side stated honestly, replica/rollup alternative raised

- 40 — describes the tradeoff accurately but won't land on an answer even after the probe
- 0 — recommends the three indexes with no cost, or claims index write cost is negligible

DEPENDS ON none

Q-LC-01

LIVECODE · WEIGHT 5

> ASK Write a single SQL query against `orders(id, customer_id, status, total_cents, created_at)` — about 200 million rows, with a B-tree index on `(customer_id, created_at)`. For every customer who ordered in the last 30 days, return their most recent order in that window, together with how many orders they placed in that window. It has to run on the primary inside the 200ms budget. Tell me when you're done, and I'll read it.

TIME BUDGET 20 min

MODE review-only

RUNTIME PostgreSQL 16 — no live database in this session; the query is scored by reading it. (A `runnable` kit would supply a seeded database and run the hidden tests at submission.)

STARTER FILE `livecode/Q-LC-01/solution.sql`

STARTER CONTENTS

```
-- orders(id bigint, customer_id bigint, status text, total_cents int, created_at timestamptz)
-- index: orders_customer_created_idx ON orders (customer_id, created_at)
-- Write your query below.
```

REQUIREMENTS

- [4] The `created_at` predicate is sargable — the raw column compared against a timestamp range, never wrapped in a function or cast that defeats the index
- [3] Returns exactly one row per customer, with ties at identical `created_at` broken deterministically
- [3] Single pass over the window — no correlated per-customer subquery, no query shape that runs once per customer
- [2] The order count covers the same 30-day window, not all time
- [1] The window boundary is computed once and is half-open, not a `BETWEEN` against a hardcoded end-of-day string
- [1] Readable: aliases, formatting, no dead columns

DELIBERATE TRAPS

- **Non-sargable predicate.** The obvious phrasing is `date(created_at) >= current_date - 30` or a cast to date. It defeats the index and is the same failure as Q-L2-01 — this question tests whether they apply that knowledge when writing, not just when diagnosing. A candidate who missed Q-L2-01 and gets this right learns the concept fast; one who got Q-L2-01 right and writes it wrong here recognises the pattern but hasn't internalised it.
- **Ties.** Two orders with identical `created_at` for one customer. `DISTINCT ON (customer_id)` without a deterministic second `ORDER BY` key returns an arbitrary row. Almost nobody handles this unprompted; it is the top-end discriminator.
- **Ambiguity, deliberately unstated:** whether cancelled orders count. The `status` column is right there in the schema and the Ask never mentions it. Asking about it earns `CLARIFIED` — that is the point of leaving it out. If asked, answer: "Count every order regardless of status." Do not volunteer this.
- **The count.** Getting the latest order *and* the window count in one pass takes a window function or an aggregate join; candidates commonly compute the count over all time by accident.

version of this kit would ship `livecode/Q-LC-01/tests.sql` with one assertion group per requirement, including a fixture row pair with identical `created_at` to catch the tie trap.

REFERENCE SOLUTION `livecode/Q-LC-01/reference.sql` — never shown.

CRAFT RUBRIC Full marks for a single `DISTINCT ON (customer_id) … ORDER BY customer_id, created_at DESC, id DESC` with a `COUNT(*) OVER (PARTITION BY customer_id)`, or an equivalent `row_number()` CTE. A correlated subquery that is otherwise correct is a −10: it answers the question and would fall over at this table size. Deduct for hardcoded literal dates, `SELECT *`, and cast-wrapped predicates. Add for a comment explaining the tie-break, which shows the trap was seen rather than stumbled past.

PROBES

- "Walk me through your query."
- "Two orders for the same customer land on the same timestamp. What comes back?"
- "How does the planner use the index for this?"
- "What would you change if this had to run every minute?"

SCORING ANCHORS

- 100 — sargable, single pass, deterministic tie-break, window-scoped count, clean
- 70 — sargable and single pass, tie-break arbitrary or count subtly wrong
- 40 — produces the right answer but with a per-customer correlated subquery, or a predicate that defeats the index
- 0 — no working query, or a shape that cannot return one row per customer

DEPENDS ON none